

WEEK 9 — AGENTIC AI & MULTI-AGENT SYSTEM DESIGN

(AutoGen + Open Models + Orchestration + Memory + Planning + Tool Use)

WEEK OBJECTIVES

Interns will learn:

Agentic system architecture

Single-agent → multi-agent systems

Role-based intelligence modelling

Planner–Executor–Validator patterns

Tool-using agents (code, search, DB, files)

Multi-agent communication protocols

Memory (short + long term)

Swarm + hierarchical orchestration

Fully local AI task automation systems

This week produces engineers who can **build autonomous AI systems (not just chatbots)**.

TECH STACK — FIXED (NO SUBSTITUTIONS)

Frameworks

- AutoGen (Microsoft)
- TinyLlama / Phi-3 / Mistral / Qwen
- FAISS (local memory)
- FastAPI
- SQLite
- Docker (optional)

Concepts

- Message passing
 - State Machine
 - Blackboard memory
 - Chain-of-thought isolation
 - Role control
 - Task delegation
-

DAY 1 — AGENT FOUNDATIONS + MESSAGE-BASED COMMUNICATION

♦ Learning Outcomes

What is an AI agent?

Agent vs chatbot vs pipeline

Perception → Reasoning → Action loop

Message protocol systems

♦ Topics to Learn

- Agent architecture
- ReAct pattern (Reason + Act)
- LLM as a tool executor
- System prompts for agents
- Role isolation

♦ Exercise

Build **3 single agents**:

1. Research Agent
2. Summarizer Agent
3. Answer Agent

Each agent must have:

- Unique role
- Unique system prompt
- Memory window = 10
- Strict job separation

Test conversation:

User → Research Agent → Info

Research Agent → Summarizer Agent → Summary

Summarizer Agent → Answer Agent → Final Answer

♦ Deliverables

/agents/research_agent.py
/agents/summarizer_agent.py
/agents/answer_agent.py
AGENT-FUNDAMENTALS.md

DAY 2 — MULTI-AGENT ORCHESTRATION (PLANNER → WORKERS → VALIDATOR)

♦ Learning Outcomes

Designing agent hierarchies
Task planning
Delegation logic
Chain-of-command structure

◆ Topics

- Planner–Executor architecture
- DAG-based execution
- Task graph generation
- Agent registry pattern

◆ Exercise

Implement **4-Agent Architecture**:

1. **Orchestrator / Planner**
2. **Worker Agent**
3. **Reflection Agent**
4. **Validator Agent**

Flow:

User Query



Orchestrator (creates steps)



Worker Agents (parallel)



Reflection Agent (improves answer)



Validator (checks for errors)



Final Answer

- ✓ Must support parallel workers
- ✓ Must show execution tree

◆ Deliverables

/orchestrator/planner.py
/agents/worker_agent.py
/agents/validator.py
FLOW-DIAGRAM.md

DAY 3 — TOOL-CALLING AGENTS (CODE, FILES, DATABASE, SEARCH)

◆ Learning Outcomes

Agents using real tools
Function calling without API
System-to-tool execution

◆ Topics

- Python tool calling
- Shell command tools
- SQLite / CSV querying
- File reading/writing
- Local search engine

◆ Exercise

Create 3 tool-using agents:

Agent	Tool
Code Agent	Python execution
DB Agent	SQLite + SQL
File Agent	Read/write .txt, .csv

Example:

User: "Analyze sales.csv and generate top 5 insights"
→ Orchestrator assigns File + Code + Analysis Agents
→ Combined final answer

♦ Deliverables

/tools/code_executor.py
/tools/db_agent.py
/tools/file_agent.py
TOOL-CHAIN.md

DAY 4 — MEMORY SYSTEMS (SHORT-TERM + LONG-TERM + VECTOR MEMORY)

♦ Learning Outcomes

Agent remembers context
Persistent memory
Similarity-based recall

♦ Topics

- Short-term (session memory)
- Long-term (SQLite)
- Vector memory (FAISS)
- Episodic vs semantic memory

♦ Exercise

Implement **Agent Memory System**:

- ✓ Conversation stored
- ✓ Important facts summarized

- ✓ Stored in FAISS
- ✓ Recall when needed

Flow:

New Query

- Search memory
- Fetch similar context
- Inject in prompt
- Generate with context

♦ Deliverables

```
/memory/session_memory.py
/memory/vector_store.py
/memory/long_term.db
MEMORY-SYSTEM.md
```

DAY 5 — CAPSTONE: AUTONOMOUS MULTI-AGENT AI SYSTEM

♦ Learning Outcomes

Full autonomous agent system
Ready for production pipelines

♦ Tasks

Build **Master Agent System** called:

PROJECT: NEXUS AI

Capabilities:

- ✓ Multi-agent orchestration
- ✓ Tool use
- ✓ Memory recall
- ✓ Self-reflection
- ✓ Self-improvement

- ✓ Multi-step planning
- ✓ Role switching
- ✓ Logs + Tracing
- ✓ Failure recovery

Example tasks it must solve:

1. "Plan a startup in AI for healthcare"
2. "Generate backend architecture for scalable app"
3. "Analyze CSV & create business strategy"
4. "Design a RAG pipeline for 50k documents"

Must contain these agents:

Agent

Orchestrator

Planner

Researcher

Coder

Analyst

Critic

Optimizer

Validator

Reporter

♦ Deliverables

```
/nexus_ai/main.py  
/nexus_ai/config.py  
/logs/  
/README.md  
ARCHITECTURE.md
```


DEMO-VIDEO.md (optional)

FINAL-REPORT.md

WEEK COMPLETION CRITERIA

Capability

Multi Agents

Orchestrator

Tool Calling

Memory

Self-Reflection

Planning

Parallel Work

No paid APIs

Fully Local

FINAL OUTCOME

After this week, engineers can:

- ✓ Build real multi-agent AI systems
- ✓ Create planner–worker–validator loops
- ✓ Create autonomous AI workflows
- ✓ Design reasoning & memory graphs
- ✓ Replace entire teams with agent swarms
- ✓ Build AI startups/platforms/copilots

They will **NOT** be prompt engineers — they will be **AI SYSTEM ENGINEERS**.